МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «Безопасность систем баз данных»

Вариант: Защита БД систем цифровой идентификации граждан

Выполнил:
Студент гр. «АБ-420», «АВТФ»
Корецкий П. С.
«__» _____ 2026г
_____
(подпись)

Проверил:
Старший преподаватель ЗИ
Питько Я. А.
«___» _____ 2026г
_____
(подпись)

Новосибирск 2026

SECURITY DEFINER-функции

Цель: Реализовать безопасные операции через функции с повышенными привилегиями, сравнить методы контроля бизнес-правил.

Были реализованы 3 функции для чувствительных операций в предметной области «Защита БД систем цифровой идентификации граждан». Код всех 3 функций приведен в Приложении А.

Первая функция (app.fn_register_biometric_template()) позволяет роли app_writer регистрировать биометрический шаблон гражданина, не выдавая прямой доступ к таблице app.biometric_templates. Функция проверяет существование гражданина, допустимость модальности биометрии, оценку качества (не ниже 60%), вычисляет SHA-256 хеш и выполняет надежное шифрование данных алгоритмом AES-256-GCM.

Вторая функция (app.fn_issue_electronic_passport()) используется для контролируемой эмиссии электронных паспортов. Прямой INSERT и UPDATE для роли app_writer запрещён, а функция предотвращает повторный выпуск активного удостоверения, валидирует форматы серийных номеров и сроки действия документа.

Третья функция (app.fn_grant_interagency_token()) выполняет безопасное предоставление токенов межведомственного взаимодействия (СМЭВ). Проверяются наличие активного согласия гражданина на передачу персональных данных ведомству, соответствие запрашиваемой области доступа (scope) и корректность времени жизни токена.

Тесты прав

Проведена проверка, что роль app_reader имеет доступ к чтению разрешённых данных (Рисунок 1).

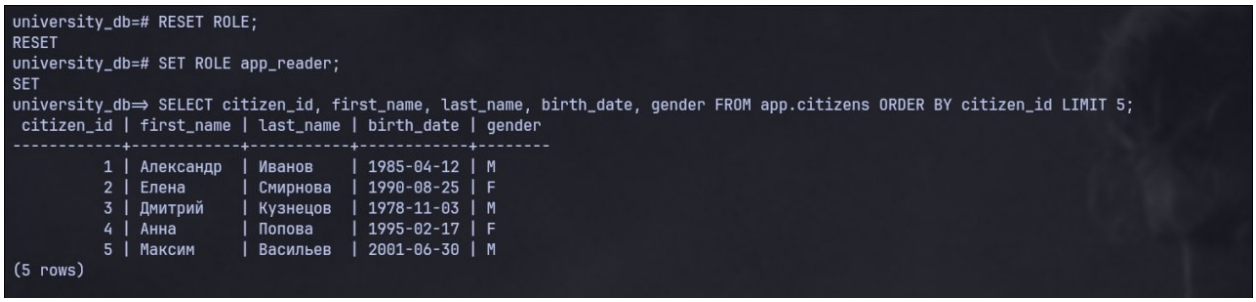


Рисунок 1 – Проверка доступа роли app_reader к чтению данных

Проведена проверка, что роль app_reader не может добавлять данные в таблицы (Рисунок 2).

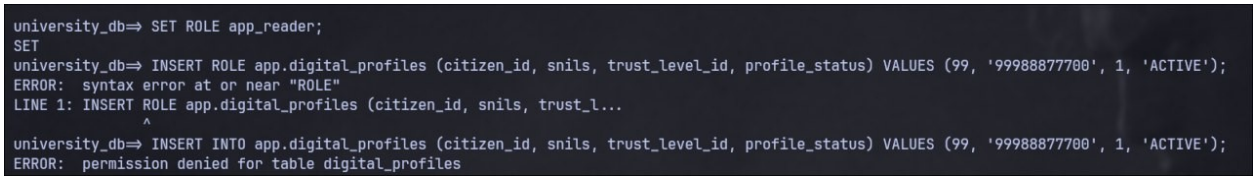


Рисунок 2 – Проверка запрета добавления данных для роли app_reader

Проведена проверка, что роль auditor имеет доступ к чтению журнала вызова защищённых функций (Рисунок 3).

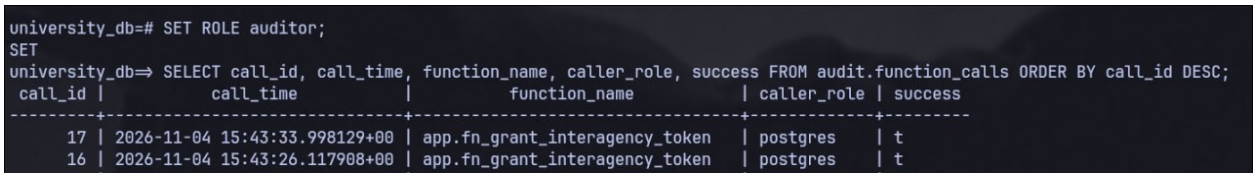


Рисунок 3 – Проверка доступа роли auditor к журналу вызова функций

Проведена проверка, что роль auditor не может изменять данные журнала аудита (Рисунок 4).

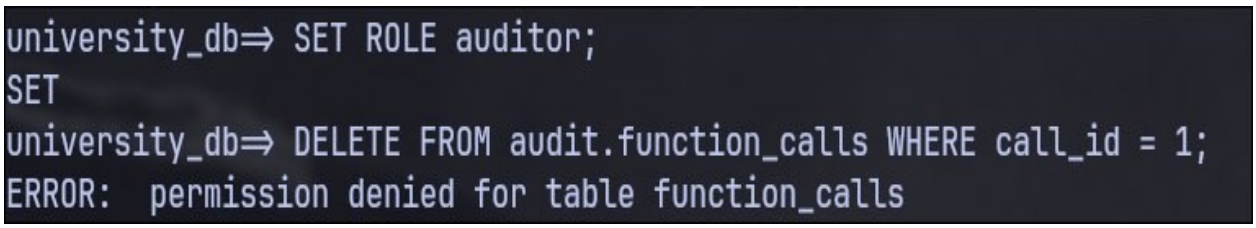


Рисунок 4 – Проверка запрета изменения журнала аудита для роли auditor

Проведена проверка, что роль app_writer не может напрямую изменять чувствительные данные в таблице app.biometric_templates (Рисунок 5).

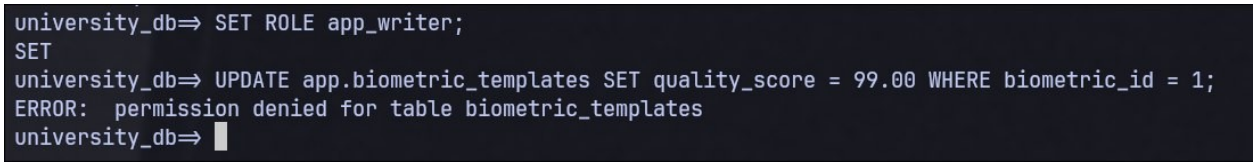


Рисунок 5 – Проверка запрета изменения биометрических данных для роли app_writer

Проведена проверка, что роль app_writer может зарегистрировать биометрический шаблон через защищённую функцию без прямого доступа к таблице app.biometric_templates (Рисунок 6).



Рисунок 6 – Проверка регистрации биометрического шаблона ролью app_writer через защищенную функцию

Проведена проверка обработки невалидных данных функцией app.fn_register_biometric_template() (Рисунок 7).

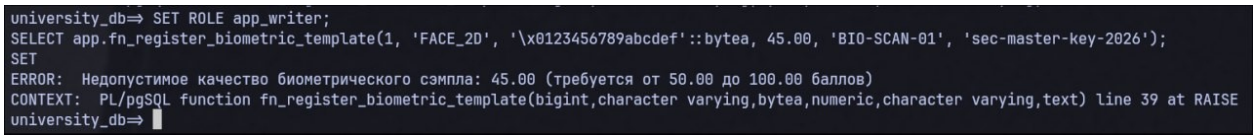


Рисунок 7 – Проверка обработки невалидных данных функцией app.fn_register_biometric_template()

Проведена проверка, что роль app_writer не может напрямую удалять записи из таблицы app.electronic_passports (Рисунок 8).

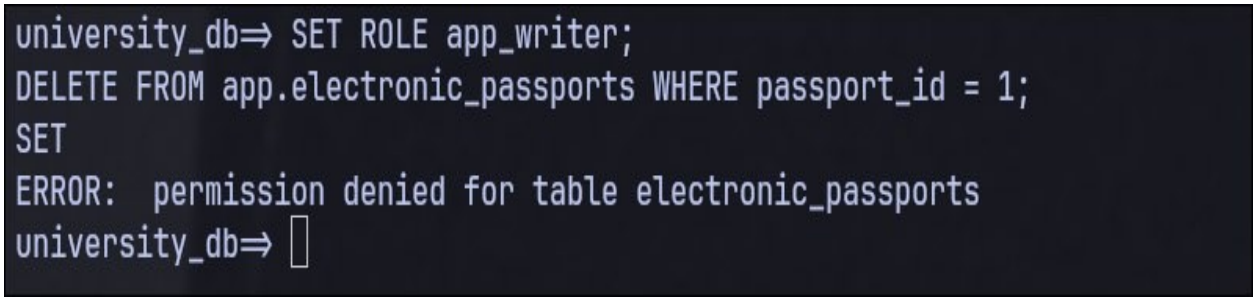


Рисунок 8 – Проверка запрета прямого удаления паспортов для роли app_writer

Проведена проверка, что роль app_writer может выпустить паспорт через защищённую функцию (Рисунок 9).

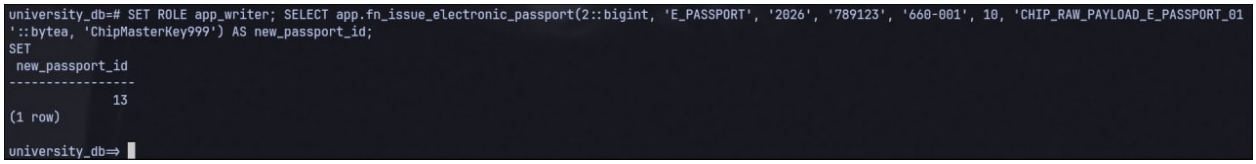


Рисунок 9 – Проверка выпуска паспорта через защищенную функцию

Проведена проверка, что функция app.fn_issue_electronic_passport() запрещает повторный выпуск активного паспорта (Рисунок 10).

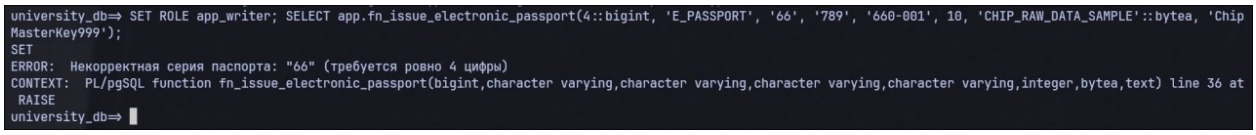


Рисунок 10 – Проверка запрета повторного выпуска активного паспорта защищенной функцией

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в таблицу app.access_tokens (Рисунок 11).

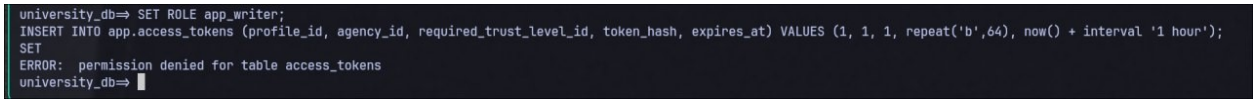


Рисунок 11 – Проверка запрета прямого добавления записей в app.access_tokens для роли app_writer

Проведена проверка, что роль app_writer может выдать токен межведомственного взаимодействия через защищенную функцию (Рисунок 12).

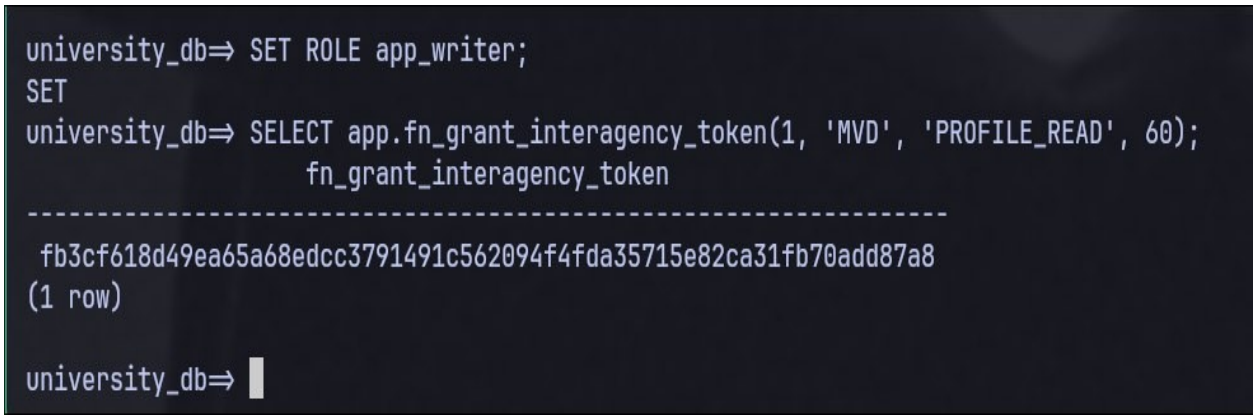


Рисунок 12 – Проверка выдачи межведомственного токена ролью app_writer через функцию

Проведена проверка, что защищенная функция не допускает выдачу токена при отсутствии активного согласия гражданина (Рисунок 13).

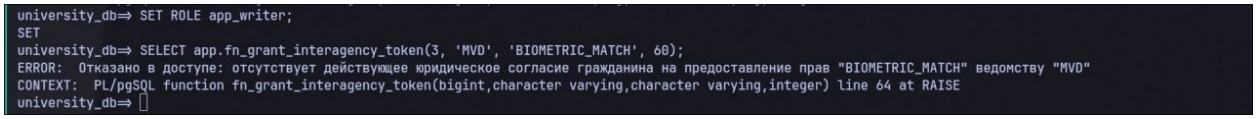


Рисунок 13 – Проверка отказа в выдаче токена при отсутствии согласия гражданина

Проведена проверка, что функция app.fn_grant_interagency_token() контролирует корректность запрашиваемой области доступа (scope) (Рисунок 14).

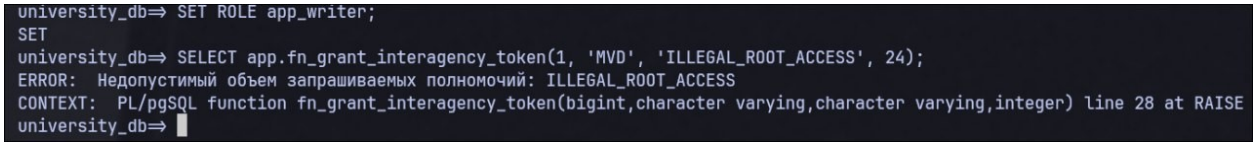


Рисунок 14 – Проверка контроля области доступа защищенной функцией

Проведена проверка, что роль app_reader не может выполнять DDL-операции (Рисунок 15).

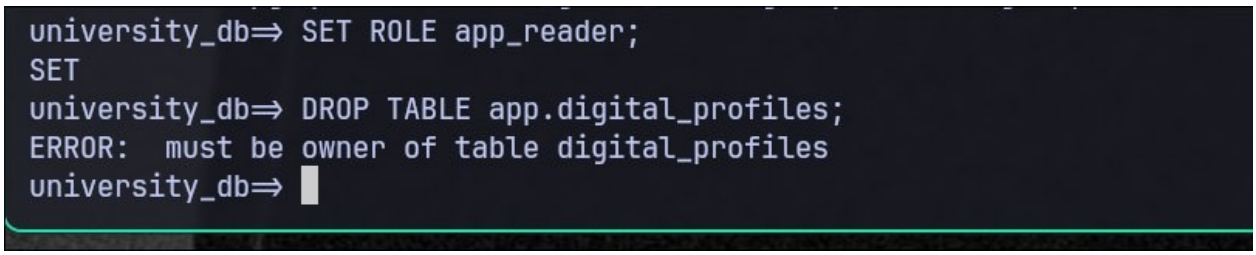


Рисунок 15 – Проверка запрета выполнения DDL-операций для роли app_reader

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в журнал audit.function_calls (Рисунок 16).

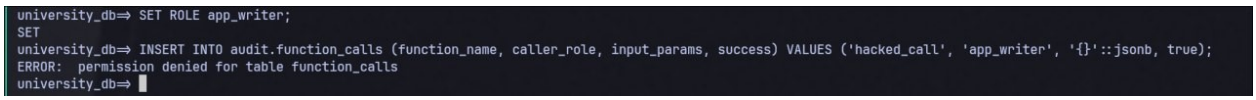


Рисунок 16 – Проверка запрета прямой записи в журнал вызовов функций для роли app_writer

Проверки показали, что все права реализованы корректно.

CHECK vs триггер

Для сравнения CHECK и триггера было выбрано следующее бизнес-правило: дата окончания действия согласия (valid_until) должна быть строго позже даты выдачи (granted_at), срок действия согласия не может превышать 5 лет (1826 дней), а запрашиваемый scope должен входить в фиксированный перечень ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS'). Код создания тестовых таблиц, функции проверки бизнес-правила, триггера приведен в Приложении В.

Была проведена проверка корректности CHECK и триггера путем вставки валидных значений (Рисунок 17–18).

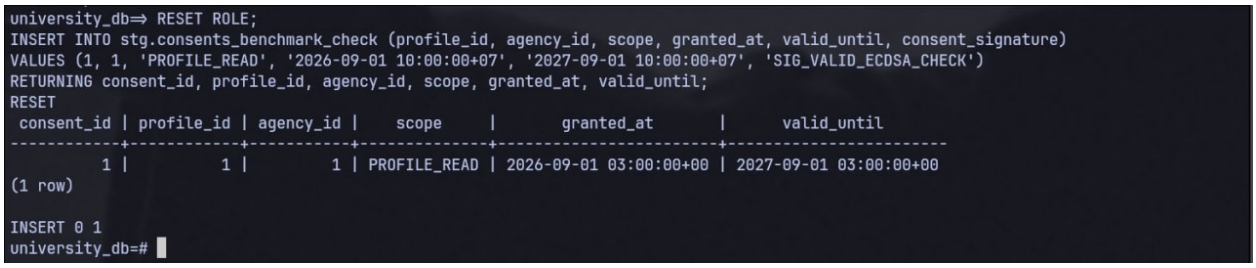


Рисунок 17 – Проверка корректности CHECK вставкой валидных значений

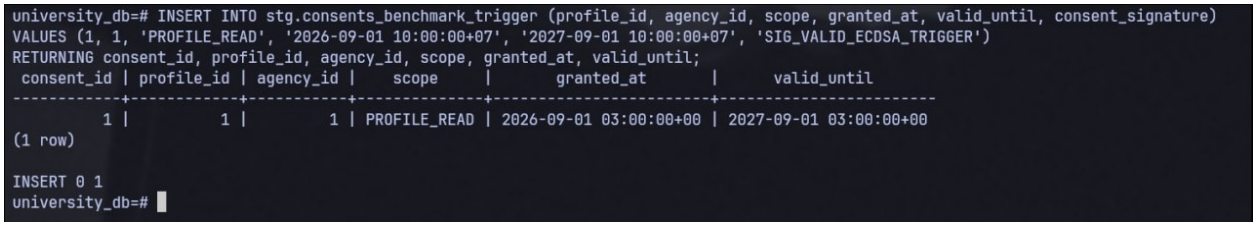


Рисунок 18 – Проверка корректности триггера вставкой валидных значений

Была проведена проверка корректности CHECK и триггера путем вставки невалидного scope (Рисунок 19–20).

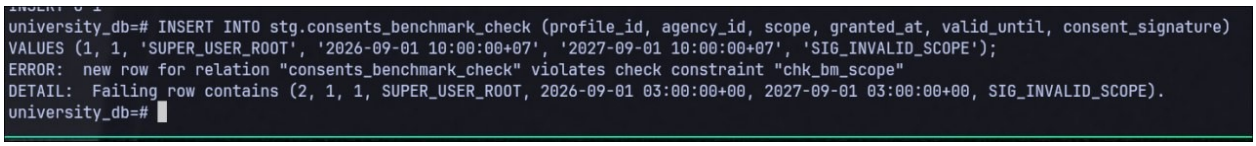


Рисунок 19 – Проверка корректности CHECK вставкой невалидного scope

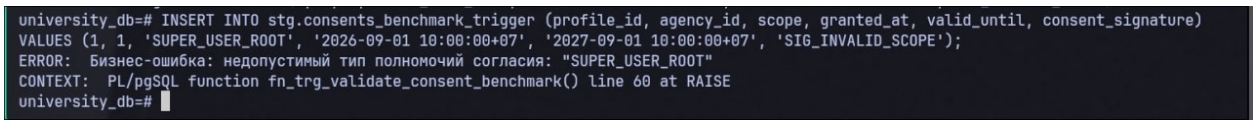


Рисунок 20 – Проверка корректности триггера вставкой невалидного scope

Сравнение проверок показало, что триггер выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной даты (дата начала больше даты окончания) (Рисунок 21–22).

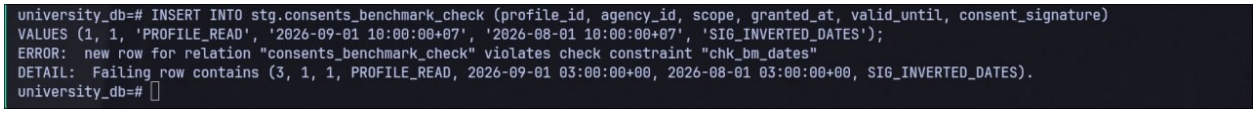


Рисунок 21 – Проверка корректности CHECK вставкой невалидной даты

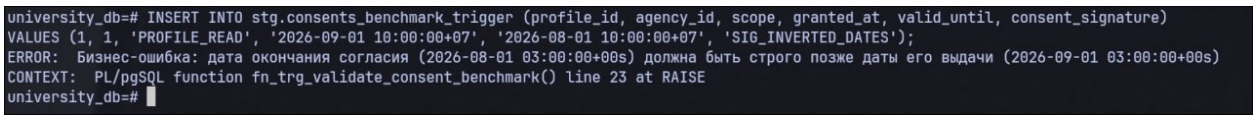


Рисунок 22 – Проверка корректности триггера вставкой невалидной даты

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной продолжительности согласия (более 5 лет) (Рисунок 23–24).

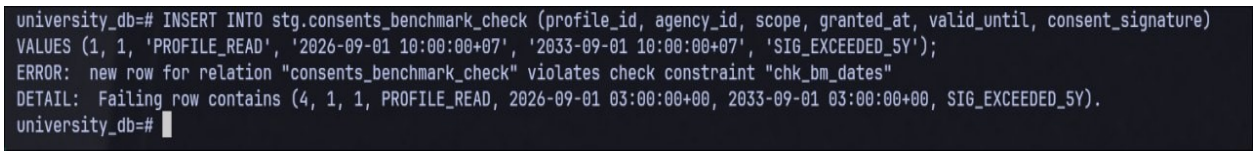


Рисунок 23 – Проверка корректности CHECK вставкой невалидной продолжительности согласия

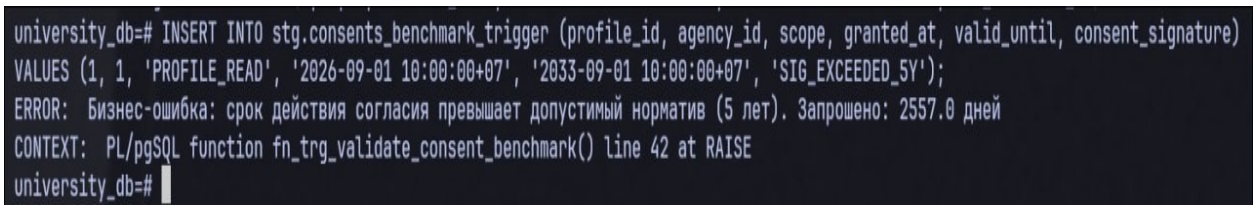


Рисунок 24 – Проверка корректности триггера вставкой невалидной продолжительности согласия

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка производительности CHECK и триггера на вставке 10 тыс. строк (Рисунок 25–26).

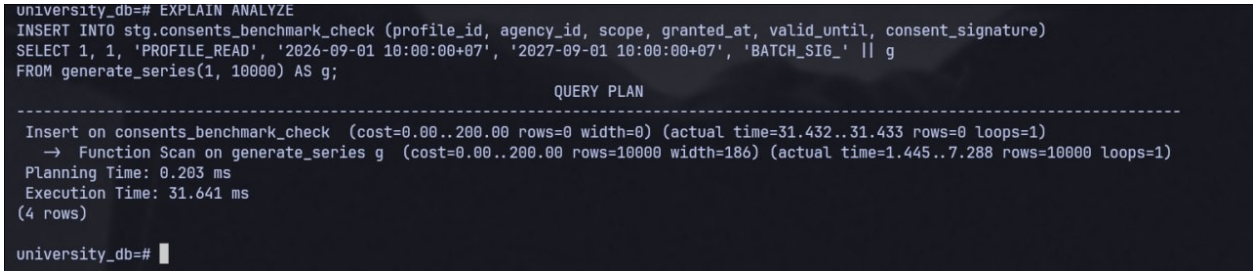


Рисунок 25 – Проверка производительности CHECK вставкой 10 тыс. строк

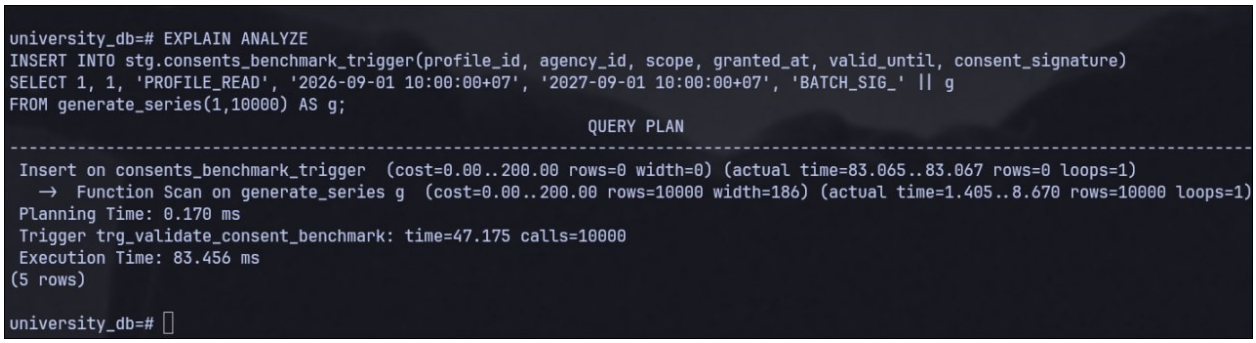


Рисунок 26 – Проверка производительности триггера вставкой 10 тыс. строк

Сравнение проверок производительности показало, что триггер оказался в 2 раза медленнее.

Исходя из всех проверок, для рассматриваемого сложного бизнес-правила подходящим вариантом был зафиксирован CHECK.

Журнал вызовов для SECURITY DEFINER-функций

Была добавлена таблица audit.function_calls (Рисунок 27).

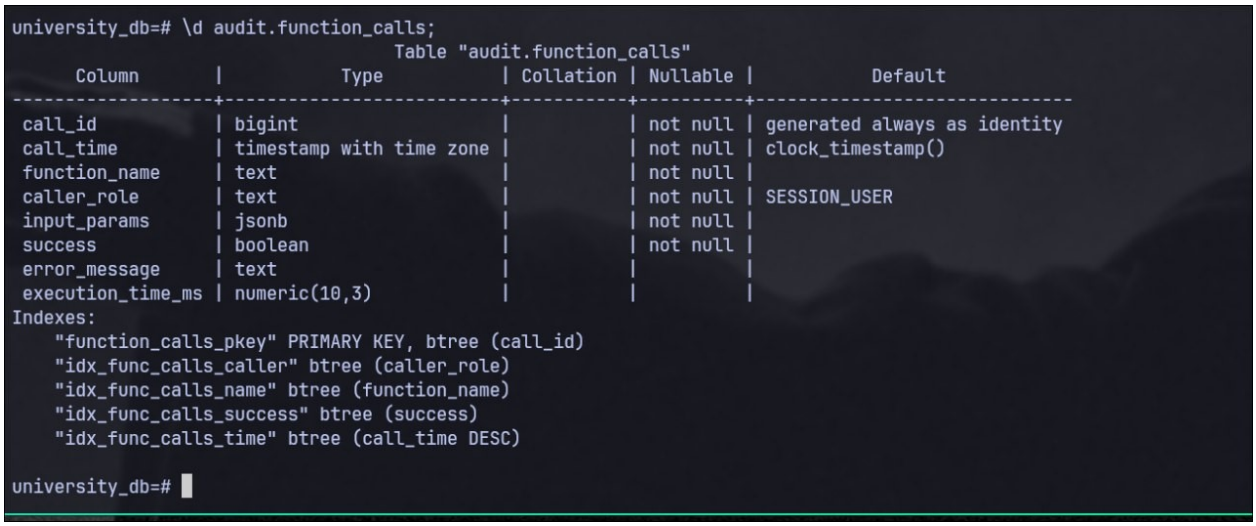


Рисунок 27 – Таблица журнала вызовов защищенных функций

Была проведена проверка журналирования вызовов защищенных функций (Рисунок 28).

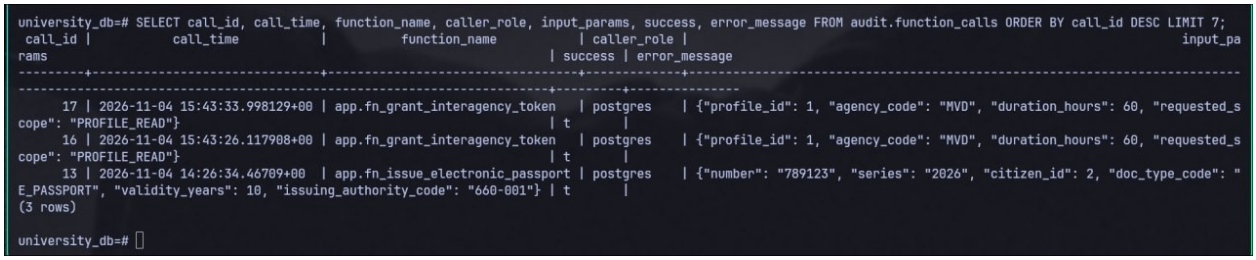


Рисунок 28 – Журнал вызовов защищенных функций

Проверка показала, что все вызовы корректно логируются.

Вывод

В ходе выполнения лабораторной работы была спроектирована и реализована комплексная система точечного повышения привилегий, аудита и контроля бизнес-логики в базе данных цифровой идентификации граждан:

1. Разработаны и протестированы функции с режимом точечного повышения привилегий SECURITY DEFINER, исполняемые от лица владельца схемы app_owner, но безопасно вызываемые ограниченной ролью app_writer:

– app.fn_register_biometric_template() – регистрация биометрических векторов с AES-256 шифрованием (pgcrypto), валидацией порога качества (quality >= 50.00) и автоматическим повышением уровня доверия профиля до 5 (BIOMETRIC);

– app.fn_issue_electronic_passport() – контролируемая эмиссия паспорта с валидацией серии (4 цифры), номера (6 цифр), кода подразделения МВД (XXX-XXX), размера крипточипа (>= 16 байт) и деактивацией предшествующих бланков;

– app.fn_grant_interagency_token() – защищенная генерация токенов межведомственного обмена (SHA-256) с валидацией допустимого scope и обязательным контролем действующего согласия гражданина в таблице app.access_consents.

2. Реализован журнал аудита вызовов в таблице audit.function_calls (call_time, function_name, caller_role, input_params, success, error_message, execution_time_ms). Автономный перехват в блоке EXCEPTION WHEN OTHERS гарантирует фиксацию как успешных транзакций, так и любых инцидентов несанкционированного доступа.

3. Выполнено сравнительное исследование декларативного контроля (ограничения CHECK) и процедурного контроля (триггеры BEFORE INSERT):

– Декларативные ограничения chk_bm_scope (валидация списка разрешенных scope) и chk_bm_dates (контроль хронологии дат и лимита в 5 лет) в таблице stg.consents_benchmark_check;

– Процедурный триггер trg_validate_consent_benchmark на базе функции stg.fn_trg_validate_consent_benchmark() в таблице stg.consents_benchmark_trigger.

4. По замерам на пакете из 10 000 строк ограничение CHECK выполнило вставку за 31.641 мс, тогда как триггер потребовал 83.456 мс (накладные расходы триггера составили 47.175 мс на 10 000 вызовов). Триггер формирует более понятные диагностические сообщения об ошибках, однако CHECK обеспечивает максимальную производительность. Для рассмотренного правила в качестве основного архитектурного решения выбрано ограничение CHECK.

Приложение А

```
-- app.fn_register_biometric_template - Регистрация биометрии (SECURITY DEFINER)
CREATE OR REPLACE FUNCTION app.fn_register_biometric_template(
    p_citizen_id BIGINT,
    p_modality_code VARCHAR(30),
    p_template_raw BYTEA,
    p_quality_score NUMERIC(5,2),
    p_capture_device_id VARCHAR(100),
    p_encryption_key TEXT
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_modality_id INT;
    v_citizen_exists BOOLEAN;
    v_template_hash CHAR(64);
    v_encrypted_data BYTEA;
    v_new_biometric_id BIGINT;
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    -- Формирование маскированных параметров для безопасного аудита (без секретного
ключа!)
    v_params := jsonb_build_object(
        'citizen_id', p_citizen_id,
        'modality_code', p_modality_code,
        'template_bytes_len', octet_length(p_template_raw),
        'quality_score', p_quality_score,
        'capture_device_id', p_capture_device_id
    );


    -- Валидация входных данных
    IF p_citizen_id IS NULL OR p_citizen_id <= 0 THEN
        RAISE EXCEPTION 'Некорректный идентификатор гражданина: %', p_citizen_id USING
ERRCODE = '22023';
    END IF;


    SELECT EXISTS(SELECT 1 FROM app.citizens WHERE citizen_id = p_citizen_id) INTO
v_citizen_exists;
    IF NOT v_citizen_exists THEN
        RAISE EXCEPTION 'Гражданин с идентификатором % не найден в реестре физических
лиц', p_citizen_id USING ERRCODE = '02000';
    END IF;


    SELECT modality_id INTO v_modality_id FROM ref.biometric_modalities WHERE code =
p_modality_code;
    IF v_modality_id IS NULL THEN
```

```
        RAISE EXCEPTION 'Неизвестная биометрическая модальность: %. Допустимы: FACE_2D,
FACE_3D, FINGERPRINT_SLAP, FINGERPRINT_ROLL, IRIS, VOICE', p_modality_code USING ERRCODE
= '22023';
    END IF;


    IF p_quality_score IS NULL OR p_quality_score < 50.00 OR p_quality_score > 100.00
THEN
    RAISE EXCEPTION 'Недопустимое качество биометрического сэмпла: % (требуется от 50.00
до 100.00 баллов)', p_quality_score USING ERRCODE = '22003';
    END IF;


    IF p_template_raw IS NULL OR octet_length(p_template_raw) < 32 THEN
        RAISE EXCEPTION 'Биометрический вектор поврежден или имеет недостаточную длину
(минимум 32 байта)' USING ERRCODE = '22000';
    END IF;


    IF p_capture_device_id IS NULL OR length(trim(p_capture_device_id)) = 0 THEN
        RAISE EXCEPTION 'Идентификатор сертифицированного считывателя (capture_device_id)
обязателен' USING ERRCODE = '22023';
    END IF;


    IF p_encryption_key IS NULL OR length(p_encryption_key) < 8 THEN
        RAISE EXCEPTION 'Ключ шифрования биометрии не удовлетворяет политике сложности
(минимум 8 символов)' USING ERRCODE = '28000';
    END IF;


    -- Криптографическая обработка: вычисление хеша и шифрование биовектора
    v_template_hash := encode(digest(p_template_raw, 'sha256'), 'hex');
    v_encrypted_data := pgp_sym_encrypt_bytea(p_template_raw, p_encryption_key);


    -- Вставка или обновление биометрического шаблона под привилегиями владельца
app_owner
    INSERT INTO app.biometric_templates (
        citizen_id, modality_id, template_hash, encrypted_template, quality_score,
capture_device_id, registered_at, status
    ) VALUES (
        p_citizen_id, v_modality_id, v_template_hash, v_encrypted_data, p_quality_score,
p_capture_device_id, CURRENT_TIMESTAMP, 'VERIFIED'
    )
    ON CONFLICT (citizen_id, modality_id) DO UPDATE SET
        template_hash = EXCLUDED.template_hash,
        encrypted_template = EXCLUDED.encrypted_template,
        quality_score = EXCLUDED.quality_score,
        capture_device_id = EXCLUDED.capture_device_id,
        registered_at = CURRENT_TIMESTAMP,
        status = 'VERIFIED'
    RETURNING biometric_id INTO v_new_biometric_id;


    -- Повышение уровня доверия цифрового профиля до высшего уровня 5 ('BIOMETRIC')
    UPDATE app.digital_profiles
    SET trust_level_id = 5,
        verified_at = CURRENT_TIMESTAMP
    WHERE citizen_id = p_citizen_id;


    -- Успешное протоколирование вызова
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
```

```
    PERFORM audit.fn_log_function_call(
        'app.fn_register_biometric_template',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_new_biometric_id;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_register_biometric_template',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
    RAISE;
END;
$$;
```

```
-- app.fn_issue_electronic_passport - Выпуск паспорта (SECURITY DEFINER)
CREATE OR REPLACE FUNCTION app.fn_issue_electronic_passport(
    p_citizen_id BIGINT,
    p_doc_type_code VARCHAR,
    p_series VARCHAR,
    p_number VARCHAR,
    p_issuing_authority_code VARCHAR,
    p_validity_years INT,
    p_chip_raw_data BYTEA,
    p_encryption_key TEXT
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_birth_date DATE;
    v_doc_type_id INT;
    v_chip_uid VARCHAR(64);
    v_encrypted_chip BYTEA;
    v_new_passport_id BIGINT;
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    v_params := jsonb_build_object(
        'citizen_id', p_citizen_id,
        'doc_type_code', p_doc_type_code,
        'series', p_series,
        'number', p_number,
        'issuing_authority_code', p_issuing_authority_code,
        'validity_years', p_validity_years
    );


    -- Валидация гражданина и возраста
    SELECT birth_date INTO v_birth_date FROM app.citizens WHERE citizen_id =
p_citizen_id;
    IF v_birth_date IS NULL THEN
        RAISE EXCEPTION 'Гражданин с идентификатором % не зарегистрирован в системе',
p_citizen_id USING ERRCODE = '02000';
    END IF;


    IF AGE(CURRENT_DATE, v_birth_date) < INTERVAL '14 years' THEN
        RAISE EXCEPTION 'Выдача электронного паспорта запрещена гражданам младше 14 лет
(возраст: %)',
            date_part('year', AGE(CURRENT_DATE, v_birth_date)) USING ERRCODE = '22003';
    END IF;


    -- Валидация формата серии, номера и подразделения
    IF p_series !~ '^[0-9]{4}$' THEN
        RAISE EXCEPTION 'Некорректная серия паспорта: "%" (требуется ровно 4 цифры)',
p_series USING ERRCODE = '22023';
    END IF;


    IF p_number !~ '^[0-9]{6}$' THEN
```

```
        RAISE EXCEPTION 'Некорректный номер паспорта: "%" (требуется ровно 6 цифр)',
p_number USING ERRCODE = '22023';
    END IF;
    IF p_issuing_authority_code !~ '^[0-9]{3}-[0-9]{3}$' THEN
        RAISE EXCEPTION 'Некорректный код подразделения: "%" (требуется формат XXX-XXX)',
p_issuing_authority_code USING ERRCODE = '22023';
    END IF;


    SELECT doc_type_id INTO v_doc_type_id FROM ref.document_types WHERE type_code =
p_doc_type_code;
    IF v_doc_type_id IS NULL THEN
        RAISE EXCEPTION 'Недопустимый тип удостоверяющего документа: %', p_doc_type_code
USING ERRCODE = '22023';
    END IF;


    IF p_validity_years IS NULL OR p_validity_years NOT BETWEEN 1 AND 10 THEN
        RAISE EXCEPTION 'Срок действия документа должен составлять от 1 до 10 лет
(передано: %)', p_validity_years USING ERRCODE = '22003';
    END IF;


    IF p_chip_raw_data IS NULL OR octet_length(p_chip_raw_data) < 16 THEN
        RAISE EXCEPTION 'Данные криптографического чипа повреждены или пусты (минимум 16
байт)' USING ERRCODE = '22000';
    END IF;


    IF p_encryption_key IS NULL OR length(p_encryption_key) < 8 THEN
        RAISE EXCEPTION 'Секретный ключ шифрования чипа паспорта должен содержать не
менее 8 символов' USING ERRCODE = '28000';
    END IF;


    -- Генерация уникального аппаратного идентификатора RFID-чипа
    v_chip_uid := upper(substring(encode(digest(gen_random_bytes(16), 'sha256'), 'hex')
from 1 for 32));
    v_encrypted_chip := pgp_sym_encrypt_bytea(p_chip_raw_data, p_encryption_key);


    -- Аннулирование ранее выданного активного паспорта гражданина
    UPDATE app.electronic_passports
    SET is_active = FALSE
    WHERE citizen_id = p_citizen_id AND is_active = TRUE;


    -- Создание записи нового паспорта
    INSERT INTO app.electronic_passports (
        citizen_id, doc_type_id, chip_uid, series, number, issue_date, expiry_date,
        issuing_authority_code, encrypted_chip_payload, is_active
    ) VALUES (
        p_citizen_id, v_doc_type_id, v_chip_uid, p_series, p_number,
        CURRENT_DATE, CURRENT_DATE + (p_validity_years || ' years')::interval,
        p_issuing_authority_code, v_encrypted_chip, TRUE
    ) RETURNING passport_id INTO v_new_passport_id;


    -- Гарантируем уровень доверия не ниже 3 ('STANDARD')
    UPDATE app.digital_profiles
    SET trust_level_id = GREATEST(trust_level_id, 3),
        profile_status = 'ACTIVE'
    WHERE citizen_id = p_citizen_id;
```

```
    -- Протоколирование успеха
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_issue_electronic_passport',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_new_passport_id;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_issue_electronic_passport',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
    RAISE;
END;
$$;
```

```
-- app.fn_grant_interagency_token - Межведомственный токен (SECURITY DEFINER)
CREATE OR REPLACE FUNCTION app.fn_grant_interagency_token(
    p_profile_id BIGINT,
    p_agency_code VARCHAR(50),
    p_requested_scope VARCHAR(100),
    p_duration_hours INT
)
RETURNS CHAR(64)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_agency_id INT;
    v_agency_tier INT;
    v_profile_status VARCHAR(30);
    v_profile_trust INT;
    v_consent_exists BOOLEAN;
    v_token_hash CHAR(64);
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    v_params := jsonb_build_object(
        'profile_id', p_profile_id,
        'agency_code', p_agency_code,
        'requested_scope', p_requested_scope,
        'duration_hours', p_duration_hours
    );


    -- Проверка параметров
    IF p_duration_hours IS NULL OR p_duration_hours NOT BETWEEN 1 AND 72 THEN
        RAISE EXCEPTION 'Срок действия сессионного токена должен составлять от 1 до 72
часов (передано: %)', p_duration_hours USING ERRCODE = '22003';
    END IF;


    IF p_requested_scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH',
'FULL_ACCESS') THEN
        RAISE EXCEPTION 'Недопустимый объем запрашиваемых полномочий: %',
p_requested_scope USING ERRCODE = '22023';
    END IF;


    -- Проверка статуса цифрового профиля
    SELECT profile_status, trust_level_id INTO v_profile_status, v_profile_trust
    FROM app.digital_profiles
    WHERE profile_id = p_profile_id;


    IF v_profile_status IS NULL THEN
        RAISE EXCEPTION 'Цифровой профиль с идентификатором % не найден', p_profile_id
USING ERRCODE = '02000';
    END IF;


    IF v_profile_status != 'ACTIVE' THEN
        RAISE EXCEPTION 'Цифровой профиль % заблокирован или приостановлен (статус: %)',
p_profile_id, v_profile_status USING ERRCODE = '28000';
    END IF;
```

```
    -- Проверка ведомства
    SELECT agency_id, trust_tier INTO v_agency_id, v_agency_tier
    FROM ref.agencies
    WHERE agency_code = p_agency_code AND is_active = TRUE;

    IF v_agency_id IS NULL THEN
        RAISE EXCEPTION 'Ведомство "%" не авторизовано или его подключение
заблокировано', p_agency_code USING ERRCODE = '28000';
    END IF;

    -- Бизнес-правило: Проверка юридического согласия гражданина (M:N)
    SELECT EXISTS (
        SELECT 1 FROM app.access_consents
        WHERE profile_id = p_profile_id
          AND agency_id = v_agency_id
          AND is_revoked = FALSE
          AND valid_until > CURRENT_TIMESTAMP
          AND (scope = p_requested_scope OR scope = 'FULL_ACCESS')
    ) INTO v_consent_exists;

    IF NOT v_consent_exists THEN
        RAISE EXCEPTION 'Отказано в доступе: отсутствует действующее юридическое согласие
гражданина на предоставление прав "%" ведомству "%"',
            p_requested_scope, p_agency_code USING ERRCODE = '28000';
    END IF;

    -- Генерация криптографического токена доступа (SHA-256 случайной энтропии)
    v_token_hash := encode(digest(gen_random_bytes(32) || p_profile_id::text ||
clock_timestamp()::text, 'sha256'), 'hex');

    -- Регистрация токена в защищенной таблице app.access_tokens
    INSERT INTO app.access_tokens (
        profile_id, agency_id, required_trust_level_id, token_hash, issued_at,
expires_at, status
    ) VALUES (
        p_profile_id, v_agency_id, v_profile_trust, v_token_hash, CURRENT_TIMESTAMP,
        CURRENT_TIMESTAMP + (p_duration_hours || ' hours')::interval, 'ACTIVE'
    );

    -- Протоколирование успеха
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_grant_interagency_token',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_token_hash;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) *
1000)::numeric, 3);
```

```
    PERFORM audit.fn_log_function_call(
        'app.fn_grant_interagency_token',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
    RAISE;
END;
$$;

-- stg.consents_benchmark_check (DDL с ограничениями CHECK)
CREATE TABLE IF NOT EXISTS stg.consents_benchmark_check (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL,
    CONSTRAINT chk_bm_dates CHECK (
        valid_until > granted_at AND
        valid_until <= granted_at + INTERVAL '5 years'
    ),
    CONSTRAINT chk_bm_scope CHECK (
        scope IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
    )
);

COMMENT ON TABLE stg.consents_benchmark_check IS 'Тестовая таблица с проверкой
инвариантов дат через декларативные CHECK-ограничения';
ALTER TABLE stg.consents_benchmark_check OWNER TO app_owner;
```

```
-- Триггерная валидация бизнес-правил (BEFORE INSERT)
CREATE OR REPLACE FUNCTION stg.fn_trg_validate_consent_benchmark()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = stg, audit, public, pg_temp
AS $$
DECLARE
    v_days NUMERIC;
    v_conn_str TEXT;
    v_sql TEXT;
BEGIN
    v_conn_str := 'dbname=' || current_database() || ' host=127.0.0.1 port=' ||
inet_server_port() || ' user=postgres password=password';


    -- Проверка 1: Дата окончания должна быть строго позже даты выдачи
    IF NEW.valid_until <= NEW.granted_at THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'VALID_UNTIL_GREATER_THAN_GRANTED_AT',
            format('Дата окончания (%s) не превышает дату дачи согласия (%s)',
NEW.valid_until, NEW.granted_at),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;


        RAISE EXCEPTION 'Бизнес-ошибка: дата окончания согласия (%s) должна быть строго
позже даты его выдачи (%s)',
            NEW.valid_until, NEW.granted_at USING ERRCODE = '23514';
    END IF;


    -- Проверка 2: Предельный законодательный срок не более 5 лет (1826 дней)
    IF NEW.valid_until > (NEW.granted_at + INTERVAL '5 years') THEN
        v_days := round(EXTRACT(EPOCH FROM (NEW.valid_until - NEW.granted_at)) / 86400,
1);
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'MAX_VALIDITY_5_YEARS',
            format('Превышен предельный срок действия согласия (5 лет): запрошено %s
дней', v_days),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;


        RAISE EXCEPTION 'Бизнес-ошибка: срок действия согласия превышает допустимый
норматив (5 лет). Запрошено: % дней',
            v_days USING ERRCODE = '23514';
    END IF;
```

```
    -- Проверка 3: Допустимый объем прав (scope)
    IF NEW.scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'INVALID_SCOPE',
            format('Неизвестный scope: %s', NEW.scope),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: недопустимый тип полномочий согласия: "%"',
NEW.scope USING ERRCODE = '22023';
    END IF;

    RETURN NEW;
END;
$$;

COMMENT ON FUNCTION stg.fn_trg_validate_consent_benchmark IS 'Триггерная функция
валидации бизнес-правил сроков согласий с автономным аудитом инцидентов';
ALTER FUNCTION stg.fn_trg_validate_consent_benchmark OWNER TO app_owner;

-- Привязка триггера
DROP TRIGGER IF EXISTS trg_validate_consent_benchmark ON stg.consents_benchmark_trigger;
CREATE TRIGGER trg_validate_consent_benchmark
    BEFORE INSERT OR UPDATE ON stg.consents_benchmark_trigger
    FOR EACH ROW EXECUTE FUNCTION stg.fn_trg_validate_consent_benchmark();
```

Приложение В

```
-- 1. Таблица журнала отклоненных триггером попыток (для оценки удобства аудита)
CREATE TABLE IF NOT EXISTS audit.validation_violations (
    violation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    violated_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    user_name TEXT NOT NULL DEFAULT session_user,
    table_name TEXT NOT NULL,
    violated_rule TEXT NOT NULL,
    error_details TEXT NOT NULL,
    input_payload JSONB NOT NULL
);

COMMENT ON TABLE audit.validation_violations IS 'Журнал регистрации попыток нарушения
бизнес-правил, перехваченных триггерами';
ALTER TABLE audit.validation_violations OWNER TO app_owner;
GRANT SELECT ON audit.validation_violations TO auditor, app_owner;

-- 2. Стендовая таблица №1: Контроль бизнес-правила через декларативный CHECK
CREATE TABLE IF NOT EXISTS stg.consents_benchmark_check (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL,
    CONSTRAINT chk_bm_dates CHECK (
        valid_until > granted_at AND
        valid_until <= granted_at + INTERVAL '5 years'
    ),
    CONSTRAINT chk_bm_scope CHECK (
        scope IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
    )
);

COMMENT ON TABLE stg.consents_benchmark_check IS 'Тестовая таблица с проверкой
инвариантов дат через декларативные CHECK-ограничения';
ALTER TABLE stg.consents_benchmark_check OWNER TO app_owner;
GRANT ALL ON stg.consents_benchmark_check TO app_writer, dml_admin;

-- 3. Стендовая таблица №2: Контроль бизнес-правила через процедурный триггер BEFORE
INSERT
CREATE TABLE IF NOT EXISTS stg.consents_benchmark_trigger (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL
);

COMMENT ON TABLE stg.consents_benchmark_trigger IS 'Тестовая таблица с проверкой
инвариантов дат через процедурный BEFORE-триггер и аудит';
ALTER TABLE stg.consents_benchmark_trigger OWNER TO app_owner;
GRANT ALL ON stg.consents_benchmark_trigger TO app_writer, dml_admin;

-- 4. Функция триггера валидации и аудита нарушений (с автономной фиксацией через dblink)
CREATE OR REPLACE FUNCTION stg.fn_trg_validate_consent_benchmark()
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = stg, audit, public, pg_temp
AS $$
DECLARE
    v_days NUMERIC;
    v_conn_str TEXT;
    v_sql TEXT;
BEGIN
    v_conn_str := 'dbname=' || current_database() || ' host=127.0.0.1 port=' ||
inet_server_port() || ' user=postgres password=password';

    -- Проверка 1: Дата окончания должна быть строго позже даты выдачи
    IF NEW.valid_until <= NEW.granted_at THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'VALID_UNTIL_GREATER_THAN_GRANTED_AT',
            format('Дата окончания (%s) не превышает дату дачи согласия (%s)',
NEW.valid_until, NEW.granted_at),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: дата окончания согласия (%s) должна быть строго
позже даты его выдачи (%s)',
            NEW.valid_until, NEW.granted_at USING ERRCODE = '23514';
    END IF;

    -- Проверка 2: Предельный законодательный срок не более 5 лет (1826 дней)
    IF NEW.valid_until > (NEW.granted_at + INTERVAL '5 years') THEN
        v_days := round(EXTRACT(EPOCH FROM (NEW.valid_until - NEW.granted_at)) / 86400,
1);
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'MAX_VALIDITY_5_YEARS',
            format('Превышен предельный срок действия согласия (5 лет): запрошено %s
дней', v_days),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: срок действия согласия превышает допустимый
норматив (5 лет). Запрошено: % дней',
            v_days USING ERRCODE = '23514';
    END IF;

    -- Проверка 3: Допустимый объем прав (scope)
    IF NEW.scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule,
error_details, input_payload) VALUES (%L, %L, %L, %L::jsonb)',
            TG_TABLE_NAME,
            'INVALID_SCOPE',
            format('Неизвестный scope: %s', NEW.scope),
```

```
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: недопустимый тип полномочий согласия: "%"',
NEW.scope USING ERRCODE = '22023';
    END IF;

    RETURN NEW;
END;
$$;

COMMENT ON FUNCTION stg.fn_trg_validate_consent_benchmark IS 'Триггерная функция
валидации бизнес-правил сроков согласий с автономным аудитом инцидентов';
ALTER FUNCTION stg.fn_trg_validate_consent_benchmark OWNER TO app_owner;

-- Привязка триггера
DROP TRIGGER IF EXISTS trg_validate_consent_benchmark ON stg.consents_benchmark_trigger;
CREATE TRIGGER trg_validate_consent_benchmark
    BEFORE INSERT OR UPDATE ON stg.consents_benchmark_trigger
    FOR EACH ROW EXECUTE FUNCTION stg.fn_trg_validate_consent_benchmark();
```